

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ
TRUYỀN THÔNG



BÁO CÁO NGHIÊN CỨU TỐT NGHIỆP 1

ĐỀ TÀI
Android Automotive OS

GVHD: TS.Hoàng Văn Hiệp

Sinh viên: Hoàng Đức Anh - 20235640

Hà Nội, ngày 18 tháng 5 năm 2026

Lời cảm ơn

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc tới TS. Hoàng Văn Hiệp, giảng viên hướng dẫn, người đã tận tình định hướng, chỉ bảo và góp ý cho em trong suốt quá trình thực hiện báo cáo nghiên cứu tốt nghiệp 1 với đề tài “Android Automotive OS”. Những ý kiến đóng góp và sự hướng dẫn của thầy đã giúp em có thêm nhiều kiến thức chuyên môn, nâng cao khả năng nghiên cứu, tổng hợp tài liệu và trình bày vấn đề một cách khoa học hơn.

Trong quá trình tìm hiểu và thực hiện đề tài, em nhận thấy Android Automotive OS là một nền tảng có nhiều điểm thú vị và giàu tiềm năng trong lĩnh vực công nghệ ô tô hiện đại. Khác với các hệ thống giải trí thông thường trên xe, Android Automotive OS được tích hợp trực tiếp vào hệ thống của phương tiện, cho phép tương tác sâu hơn với các chức năng như điều hướng, giải trí, kết nối, điều khiển một số thành phần của xe và quản lý trải nghiệm người dùng. Điều này giúp em hiểu rõ hơn về xu hướng kết hợp giữa công nghệ thông tin, hệ điều hành nhúng và ngành công nghiệp ô tô thông minh.

Thông qua đề tài này, em đã có cơ hội học hỏi thêm nhiều kiến thức mới, không chỉ về hệ điều hành Android mà còn về kiến trúc phần mềm trên ô tô, các tầng giao tiếp phần cứng, VHAL, Car Service, cũng như cách các ứng dụng được xây dựng và vận hành trong môi trường xe. Quá trình nghiên cứu cũng giúp em rèn luyện kỹ năng đọc tài liệu kỹ thuật, phân tích hệ thống, chọn lọc thông tin và trình bày nội dung theo hướng rõ ràng, logic hơn.

Do thời gian thực hiện và kiến thức của bản thân còn hạn chế, báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp từ thầy để có thể hoàn thiện nội dung nghiên cứu tốt hơn trong các giai đoạn tiếp theo.

Em xin chân thành cảm ơn!

Mục lục

Lời cảm ơn	1
Phần mở đầu	5
I Tổng quan về Android Automotive OS	7
I.1 Giới thiệu chung	7
I.2 Mối quan hệ giữa Android Automotive OS và Android	8
I.3 Phân biệt Android Automotive OS và Android Auto	8
II Kiến trúc hệ thống Android Automotive OS	10
II.1 Kiến trúc tổng quan	10
II.2 Vehicle Hardware Abstraction Layer	10
II.3 Car Service	10
II.4 Ứng dụng trên Android Automotive OS	10
III Triển khai ứng dụng Hust_Med_Demo	11
III.1 Đặc điểm của một Media-app trên AAOS	11
III.1.1 Đặc điểm Media-app	11
III.1.2 Các định nghĩa cần hiểu	12
III.1.3 Workflow	14
III.2 Giới thiệu Project Hust_Med_Demo	16
III.3 Cấu trúc thư mục project	17
III.3.1 Thư mục <code>app/src/main</code>	17
III.3.2 File <code>AndroidManifest.xml</code>	18
III.3.3 Các thư mục tài nguyên trong <code>res</code>	21
III.3.4 Các file mã nguồn trong <code>main</code>	22
III.4 Các thành phần chính của ứng dụng	30
III.5 Luồng hoạt động của ứng dụng	30
III.6 Kết quả triển khai	30

III.7 Khó khăn và cách khắc phục	30
IV Ứng dụng thực tế và xu hướng phát triển	31
IV.1 Ứng dụng trong ngành công nghiệp ô tô	31
IV.2 Ưu điểm	31
IV.3 Hạn chế	31
IV.4 Xu hướng phát triển	31
Kết luận	32

Danh sách hình vẽ

III.1 Workflow cơ bản của Media-app trên Android Automotive OS 15

Phần mở đầu

Lý do chọn đề tài

Trong những năm gần đây, ngành công nghiệp ô tô đang có sự chuyển dịch mạnh mẽ từ các phương tiện cơ khí truyền thống sang các phương tiện thông minh, có khả năng kết nối, xử lý dữ liệu và cung cấp trải nghiệm số cho người dùng. Các hệ thống phần mềm trên ô tô ngày càng đóng vai trò quan trọng, không chỉ phục vụ nhu cầu giải trí mà còn hỗ trợ điều hướng, kết nối, điều khiển một số chức năng của xe và nâng cao trải nghiệm người lái.

Android Automotive OS là một nền tảng hệ điều hành được phát triển dựa trên Android, thiết kế dành riêng cho các hệ thống thông tin giải trí và điều khiển trên ô tô. Khác với Android Auto, Android Automotive OS có thể hoạt động trực tiếp trên phần cứng của xe và tích hợp sâu với các thành phần của phương tiện. Điều này khiến nền tảng trở thành một hướng nghiên cứu đáng chú ý trong bối cảnh ô tô hiện đại ngày càng phụ thuộc vào phần mềm.

Vì vậy, em lựa chọn đề tài “Android Automotive OS” nhằm tìm hiểu thêm về công nghệ mới này, kiến trúc hệ thống, các thành phần chính, khả năng ứng dụng thực tế cũng như vai trò của Android Automotive OS trong xu hướng phát triển của đa số các hãng xe thông minh hiện nay.

Mục tiêu nghiên cứu

Mục tiêu của báo cáo là nghiên cứu và trình bày một cách tổng quan về Android Automotive OS, từ đó giúp làm rõ vai trò, đặc điểm và tiềm năng ứng dụng của nền tảng này trong ngành công nghiệp ô tô.

Cụ thể, báo cáo hướng tới các mục tiêu sau:

- Tìm hiểu khái niệm và đặc điểm cơ bản của Android Automotive OS.
- Phân biệt Android Automotive OS với Android Auto.
- Nghiên cứu kiến trúc tổng quan và các thành phần chính của Android Automotive OS.
- Tìm hiểu vai trò của Vehicle Hardware Abstraction Layer, Car Service và các ứng dụng trên hệ thống.
- Phân tích ưu điểm, hạn chế và xu hướng phát triển của Android Automotive OS trong thực tế.

Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của báo cáo là nền tảng Android Automotive OS, bao gồm kiến trúc phần mềm, các thành phần hệ thống, cơ chế giao tiếp với phần cứng phương tiện và các ứng dụng trong môi trường ô tô.

Phạm vi nghiên cứu tập trung chủ yếu vào khía cạnh phần mềm của Android Automotive OS. Báo cáo không đi sâu vào thiết kế phần cứng ô tô, các hệ thống điều khiển an toàn chuyên dụng hay triển khai thực tế trên từng dòng xe cụ thể. Thay vào đó, nội dung chủ yếu tập trung vào việc tìm hiểu nền tảng, kiến trúc, chức năng và tiềm năng ứng dụng của Android Automotive OS.

Phương pháp nghiên cứu

Để thực hiện báo cáo, em sử dụng phương pháp nghiên cứu tài liệu, tổng hợp và phân tích thông tin từ các nguồn tham khảo liên quan đến Android Automotive OS, hệ điều hành Android, hệ thống phần mềm trên ô tô và các công nghệ hỗ trợ xe thông minh.

Quá trình nghiên cứu bao gồm việc đọc tài liệu kỹ thuật, phân tích kiến trúc hệ thống, so sánh Android Automotive OS với các nền tảng liên quan, đồng thời tổng hợp các ưu điểm, hạn chế và xu hướng phát triển của nền tảng này trong ngành công nghiệp ô tô hiện đại.

Cấu trúc báo cáo

Ngoài phần mở đầu, lời cảm ơn, kết luận và tài liệu tham khảo, nội dung chính của báo cáo được chia thành các chương như sau:

- **Chương 1: Tổng quan về Android Automotive OS** – Trình bày khái niệm, lịch sử phát triển và sự khác biệt giữa Android Automotive OS và Android Auto.
- **Chương 2: Kiến trúc hệ thống Android Automotive OS** – Tìm hiểu kiến trúc tổng quan, các tầng phần mềm, Vehicle Hardware Abstraction Layer và Car Service.
- **Chương 3: Các chức năng và dịch vụ chính** – Trình bày các chức năng nổi bật như hệ thống thông tin giải trí, điều hướng, kết nối, quản lý người dùng và bảo mật.
- **Chương 4: Ứng dụng thực tế và xu hướng phát triển** – Phân tích ứng dụng của Android Automotive OS trong thực tế, ưu điểm, hạn chế và xu hướng phát triển trong tương lai.

Chương I. Tổng quan về Android Automotive OS

I.1 Giới thiệu chung

Android Automotive OS, thường được viết tắt là AAOS, là một hệ điều hành mã nguồn mở dựa trên nền tảng Android, được thiết kế để cài đặt và chạy **trực tiếp** trên phần cứng trung tâm điều khiển, hay còn gọi là hệ thống thông tin giải trí (*infotainment system*), của ô tô. Có thể hiểu một cách đơn giản rằng, thay vì phải kết nối với một chiếc điện thoại di động như cách truyền thống, AAOS biến màn hình trên xe ô tô thành một thiết bị Android độc lập với đầy đủ các chức năng cần thiết.

Với kiến trúc được xây dựng nhằm thay thế và cải tiến các hệ điều hành truyền thống trên ô tô, AAOS có một số đặc điểm nổi bật như sau:

- **Nền tảng full-stack:** AAOS bao phủ nhiều tầng trong hệ thống phần mềm của xe, từ tầng lõi giao tiếp trực tiếp với mạng lưới thiết bị của phương tiện như cảm biến, hệ thống điều hòa, âm thanh thông qua Vehicle HAL (*Hardware Abstraction Layer*), cho đến tầng Framework và tầng Application ở phía trên.
- **Hỗ trợ công nghệ hiện đại:** Thay vì phải làm quen với các framework HMI chuyên biệt của ngành ô tô, các nhà phát triển có thể sử dụng trực tiếp các bộ công cụ tiêu chuẩn của Android. Việc xây dựng một ứng dụng Media Service cho ô tô có thể được tối ưu hóa bằng các thư viện như **Jetpack Compose** cho phần giao diện người dùng, kết hợp với thư viện **Media3** để quản lý các phiên phát lại âm thanh hoặc video ở tầng bên dưới. So với cách phát triển truyền thống, vốn thường phải thiết kế giao diện thông qua các file `.xml` khá phức tạp và tốn nhiều thời gian, phương pháp này giúp quá trình phát triển ứng dụng trở nên linh hoạt và thuận tiện hơn.
- **Tính tùy biến:** Các nhà sản xuất xe (*OEM – Original Equipment Manufacturer*) có thể chỉnh sửa mã nguồn của AAOS để tạo ra giao diện người dùng (*HMI – Human Machine Interface*) mang phong cách và định vị thương hiệu riêng. Ví dụ, VinFast và Mazda có thể thiết kế giao diện người dùng khác với Hyundai và Ford nhằm tạo ra trải nghiệm riêng biệt cho khách hàng.
- **Google Automotive Services:** Bản thân AAOS là một nền tảng mã nguồn mở thuộc AOSP (*Android Open Source Project*). Tuy nhiên, Google cung cấp thêm một gói bản quyền có tên là Google Automotive Services, viết tắt là GAS. Gói này bao gồm các ứng dụng và dịch vụ cốt lõi như Google Play Store, Google Maps và Google Assistant. Các hãng xe lớn như Volvo, Polestar hoặc General Motors có thể sử dụng gói này để người dùng tải và sử dụng ứng dụng trực tiếp trên xe.

I.2 Mối quan hệ giữa Android Automotive OS và Android

Một điểm quan trọng khi tìm hiểu Android Automotive OS là cần hiểu rõ mối quan hệ giữa nền tảng này và hệ sinh thái Android nói chung. Android Automotive OS không phải là một hệ điều hành hoàn toàn tách biệt hay một nhánh phát triển song song với Android, mà về bản chất, **Android Automotive OS chính là Android được mở rộng cho môi trường ô tô.**

Android Automotive OS sử dụng cùng cơ sở mã nguồn với Android đang được triển khai trên các thiết bị như điện thoại thông minh, máy tính bảng và nhiều thiết bị thông minh khác. Nền tảng này được xây dựng dựa trên những thành phần cốt lõi của Android đã được phát triển và hoàn thiện trong nhiều năm, bao gồm mô hình bảo mật, chương trình tương thích, công cụ dành cho nhà phát triển và hệ sinh thái thư viện phong phú. Nhờ đó, Android Automotive OS có thể tận dụng được sự ổn định, tính linh hoạt và khả năng mở rộng của Android, đồng thời vẫn đảm bảo đặc điểm mã nguồn mở và khả năng tùy biến cao.

Bên cạnh việc kế thừa nền tảng Android, Android Automotive OS còn mở rộng Android để đáp ứng các yêu cầu đặc thù của ngành công nghiệp ô tô. Các yêu cầu này bao gồm khả năng tích hợp với hệ thống thông tin giải trí trên xe (*In-Vehicle Infotainment – IVI*), giao tiếp với phần cứng phương tiện, hỗ trợ các dịch vụ dành riêng cho ô tô và cung cấp giao diện người dùng phù hợp với ngữ cảnh sử dụng trong xe. Việc mở rộng này giúp Android Automotive OS không chỉ là một hệ điều hành hiển thị nội dung giải trí, mà còn là một nền tảng phần mềm có khả năng hỗ trợ nhiều chức năng khác nhau của phương tiện như : đo nhiệt độ xe, áp suất lốp xe, ...

Có thể nói, Android Automotive OS mang sức mạnh của Android vào môi trường ô tô, cho phép các nhà sản xuất xe xây dựng những hệ thống thông minh, có khả năng cập nhật, dễ tùy biến và phù hợp với xu hướng xe được định nghĩa bằng phần mềm (*Software-Defined Vehicle*). Đây là một hướng phát triển quan trọng trong thời đại số, khi trải nghiệm người dùng trên ô tô ngày càng phụ thuộc nhiều hơn vào phần mềm, kết nối và các dịch vụ thông minh.

I.3 Phân biệt Android Automotive OS và Android Auto

Android Automotive OS và Android Auto là hai khái niệm rất dễ gây nhầm lẫn do đều liên quan đến hệ sinh thái Android trên ô tô. Tuy nhiên, dưới góc độ kiến trúc hệ thống, hai nền tảng này có sự khác biệt rõ rệt.

- **Android Auto:** Android Auto về bản chất là một ứng dụng phản chiếu màn hình (*screen mirroring*) từ điện thoại lên màn hình trung tâm của xe. Trong mô hình này, điện thoại thông minh mới là thiết bị chính thực hiện xử lý và chạy hệ điều hành. Khi được kết nối với xe, điện thoại sẽ truyền dữ liệu hiển thị, luồng âm thanh (*audio streaming*) lên màn hình xe, đồng thời nhận lại các sự kiện điều khiển như thao tác cảm ứng (*touch events*)

từ hệ thống của xe. Do đó, nếu ngắt kết nối điện thoại, Android Auto sẽ không thể tiếp tục hoạt động.

- **Android Automotive OS (AAOS):** Android Automotive OS là một hệ điều hành hoàn chỉnh, được cài đặt và chạy độc lập trực tiếp trên chip xử lý của ô tô. Với AAOS, bản thân chiếc xe có thể tự kết nối Internet, tự cài đặt ứng dụng, tự lưu trữ dữ liệu và xử lý các tác vụ mà không cần phụ thuộc vào điện thoại thông minh. Nói cách khác, AAOS biến hệ thống thông tin giải trí trên xe thành một thiết bị Android độc lập, được tích hợp sâu với phần cứng và các dịch vụ của phương tiện.

Chương II. Kiến trúc hệ thống Android Automotive OS

II.1 Kiến trúc tổng quan

II.2 Vehicle Hardware Abstraction Layer

II.3 Car Service

II.4 Ứng dụng trên Android Automotive OS

Chương III. Triển khai ứng dụng Hust_Med_Demo

III.1 Đặc điểm của một Media-app trên AAOS

III.1.1 Đặc điểm Media-app

- **Không yêu cầu tự xây dựng UI:** Giao diện của **Media-app** phần lớn do OEM- các nhà sản xuất ô-tô, mỗi OEM có thể render khác nhau, hoặc system quyết định. Ứng dụng chỉ cần cung cấp dữ liệu như danh sách nhạc, metadata (các thông tin đi kèm của bài hát), ảnh bìa và AAOS sẽ render UI theo chuẩn chung nhằm đảm bảo sự phân tâm tối thiểu và an toàn khi lái xe.
- **Background Playback** và **Audio Focus** : Ứng dụng bắt buộc phải hoạt động dưới dạng **Foreground Service** để tiếp tục phát nhạc khi người dùng chuyển sang app khác. Đồng thời app phải tuân thủ nghiêm ngặt các quy tắc xử lý **Audio Focus** , ví dụ: tạm dừng khi có cuộc điện thoại, giảm âm lượng khi có thông báo chỉ đường.

– Thứ nhất, vì sao là **Foreground Service** ?

Media-app cần **Service** chạy nền có ưu tiên cao, được Android cho phép hoạt động lâu dài, và phải hiển thị **notification** cho người dùng để đảm bảo tính minh bạch. Nói một cách đơn giản, nếu người dùng:

- * Bật dẫn đường GPS;
- * Gọi điện;
- * Ghi âm;

⇒ Vẫn cần chức năng phát nhạc hoạt động dù đã thoát UI của **Media-app** .

Vì vậy: nếu **Media-app** muốn chạy nền lâu dài thì phải dùng **Foreground Service** .

– Thứ hai, **Audio Focus** :

Trong Android và đặc biệt là Android Automotive OS, **Audio Focus** là cơ chế quản lý: “**Ứng dụng nào được quyền phát âm thanh ở thời điểm hiện tại**”. Nói một cách đơn giản, Android dùng nó để tránh nhiều app nói cùng lúc rồi quyết định app nào phải: dừng, giảm âm lượng hoặc dừng hẳn.

Ví dụ : User đang nghe nhạc, Google Maps nói: “Turn left” ⇒ Nhạc phải nhỏ lại để Google Maps nói rõ.

- **Cấu trúc **MediaSession** và **MediaController** :** **Media-app** đóng vai trò như một Server cung cấp **MediaSession** . Hệ thống ô tô đóng vai trò là Client thông qua **MediaController** , liên tục gửi các lệnh điều khiển như **Play** , **Pause** , **Next** , **Browse** tới Server và nhận **playback state** từ **Media-app** .

III.1.2 Các định nghĩa cần hiểu

Trước khi đi vào chi tiết triển khai project, cần làm rõ một số khái niệm quan trọng trong kiến trúc của một Media-app trên Android Automotive OS.

A. MediaService

`MediaService` là service chính của `Media-app`. Đây là thành phần trung tâm chịu trách nhiệm quản lý quá trình phát nhạc và giao tiếp với hệ thống Android Automotive OS. Chức năng chính của `MediaService` bao gồm:

- Chứa và khởi tạo `MediaSession` ;
- Quản lý `ExoPlayer` ;
- Cung cấp danh sách bài hát;
- Xử lý `playback` ;
- Giao tiếp với AAOS.

B. ExoPlayer

`ExoPlayer` là thư viện phát media của Android do Google phát triển. Chức năng chính của nó là : **phát nội dung audio hoặc video trong ứng dụng.**

C. MediaSession

`MediaSession` là một object đại diện cho media player của app và cho phép hệ thống điều khiển nó. Về cơ bản, `MediaSession` là phần trung gian giữa bộ phát nhạc thật, tức `ExoPlayer`, và `MediaController`, tức bộ điều khiển. Các nhiệm vụ chính của nó là:

1. Nhận command từ `MediaController`

Ví dụ:

- `play` ;
- `pause` ;
- `next` ;
- `seek` .

2. Gửi playback-state cho `MediaController` Playback State = trạng thái hiện tại của media player

Ví dụ:

- Playing = đang phát;
- Paused = đang tạm dừng;
- ERROR = lỗi;
- Tiến trình trên seekbar (progress-bar), tức đang phát tới đâu.

★ Vì sao cần `MediaSession` ?

Vì hệ thống của AAOS không biết code của `Media-app`, và cũng không được gọi trực tiếp `ExoPlayer`. Do đó, quá trình điều khiển phải thông qua luồng:

`MediaController` → `MediaSession` → `ExoPlayer`

Đồng thời, AAOS cũng yêu cầu mọi `Media-app` phải public `MediaSession` để hệ thống có thể giao tiếp theo chuẩn chung.

D. MediaController

`MediaController` là thành phần phía client: chính là AAOS UI, dùng object `MediaController` để điều khiển `MediaSession`. Vì vậy, `MediaController` không phát nhạc trực tiếp, mà chỉ có các chức năng sau:

- Gửi command cho `MediaSession`;
- Đọc playback-state metadata, thông tin bài hát, v.v từ `MediaSession`.

Command	Ý nghĩa
<code>play()</code>	Phát nhạc
<code>pause()</code>	Tạm dừng phát nhạc
<code>seekTo()</code>	Tua tới một vị trí cụ thể trong bài hát
<code>skipToNext()</code>	Chuyển sang bài tiếp theo
<code>setMediaItem()</code>	Chọn bài hát hoặc nội dung media cần phát

Bảng III.1: Một số command phổ biến của MediaController

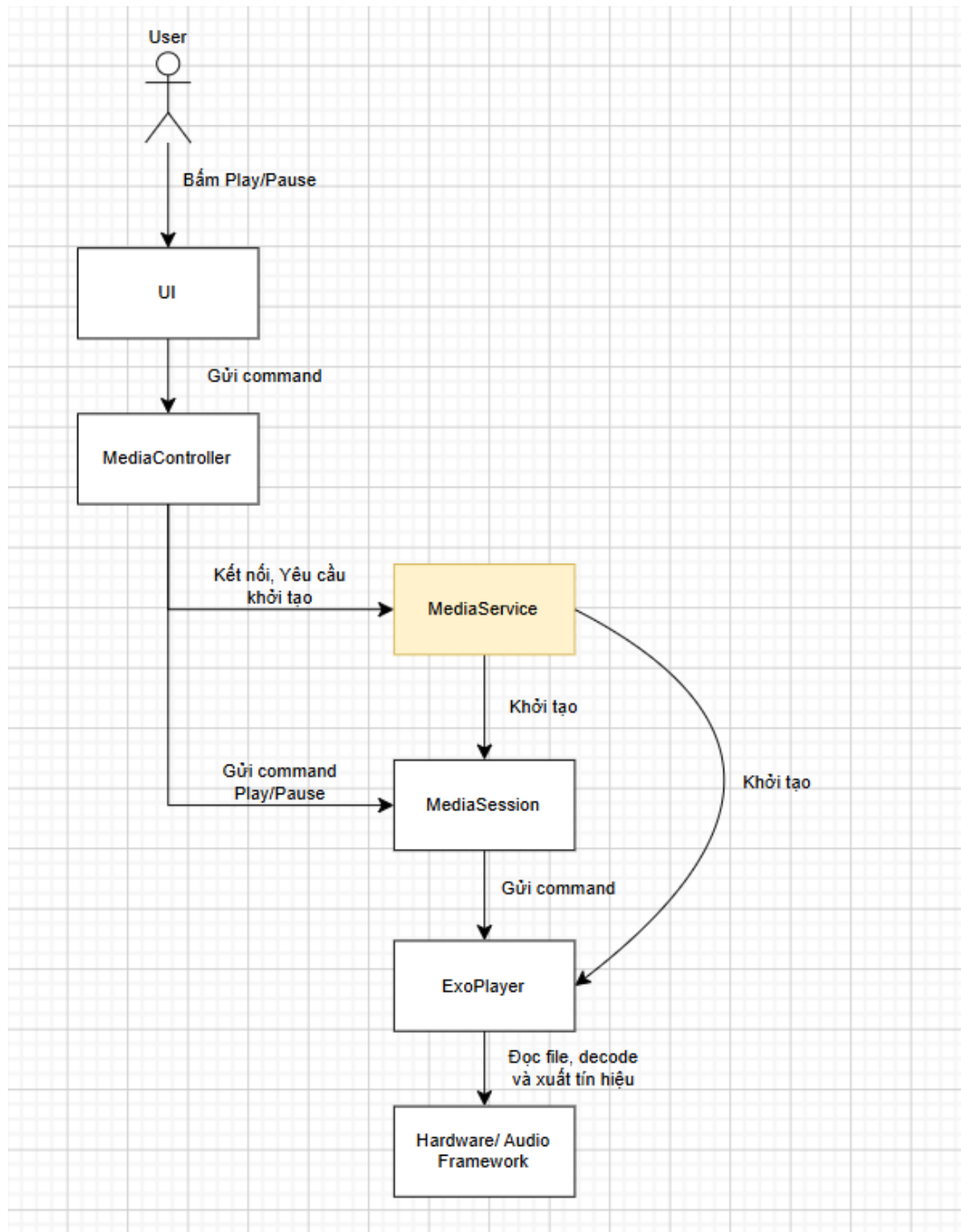
Tóm tắt vai trò các thành phần

Thành phần	Vai trò
MediaService	Chứa MediaSession, quản lý ExoPlayer, cung cấp danh sách bài hát
MediaController	Client gửi command
MediaSession	Server nhận command
ExoPlayer	Playback engine, chịu trách nhiệm phát video, nhạc

Bảng III.2: Vai trò của các thành phần chính trong Media-app

III.1.3 Workflow

Workflow cơ bản của **Media-app** có thể được mô tả thông qua sơ đồ sau:



Hình III.1: Workflow cơ bản của Media-app trên Android Automotive OS

Giải thích Workflow theo các bước

Luồng hoạt động cơ bản của **Media-app** có thể được giải thích theo các bước sau:

1. Kết nối ban đầu:

MediaController của xe tìm đến ứng dụng thông qua **MediaService**. Lúc này, **MediaService** khởi tạo sẵn **ExoPlayer** và **MediaSession**, sau đó trả về **MediaLibrarySession**.

2. Giao tiếp trực tiếp:

Kể từ thời điểm kết nối thành công, `MyMediaService` chỉ đóng vai trò giữ cho ứng dụng không bị hệ thống tắt nhờ chạy dưới dạng `Foreground Service`. Mọi giao tiếp duyệt nhạc hay điều khiển `Play`, `Pause`, `Next` giờ đây diễn ra trực tiếp giữa `MediaController` của xe và `MediaLibrarySession` của app. Điều này có nghĩa là `MediaService` chỉ khởi tạo 1 lần và về sau sẽ chỉ còn `MediaController` giao tiếp với `MediaSession`

3. Phân nhánh xử lý lệnh:

Tùy theo loại lệnh được gửi từ hệ thống, `MediaLibrarySession` sẽ xử lý theo các hướng khác nhau:

- Nếu lệnh là xin danh sách bài hát, `MediaLibrarySession` gọi tới `AutomotiveLibraryCallback` để lấy dữ liệu.
- Nếu lệnh là `Play`, `Pause` hoặc `Seek`, `MediaLibrarySession` sẽ tự động chuyển lệnh đó thẳng vào `ExoPlayer` mà không cần code trung gian.

4. Xuất âm thanh:

`ExoPlayer` đọc file âm thanh, giải mã và xuất tín hiệu ra loa của xe thông qua Android Audio Framework.

III.2 Giới thiệu Project Hust_Med_Demo

Hust_Med_Demo là một ứng dụng phát nhạc, hướng tới các hệ thống thông tin giải trí trên ô tô (*In-Vehicle Infotainment – IVI*). Project này được phát triển để minh họa cách một **Media-app** có thể tích hợp với các dịch vụ hệ thống của Android Automotive OS, từ đó cung cấp khả năng phát âm thanh, điều khiển nội dung và tương tác với giao diện Media Center của AAOS.

Thông qua Hust_Med_Demo, người dùng có thể trải nghiệm các chức năng cơ bản của một ứng dụng phát nhạc trên môi trường ô tô như hiển thị danh sách nhạc, thông tin bài hát, phát nhạc, tạm dừng, tua nhạc. Điểm quan trọng của project không chỉ nằm ở chức năng phát nhạc, mà còn ở việc ứng dụng có thể hoạt động phù hợp với kiến trúc media của Android Automotive OS, cho phép hệ thống nhận diện và hiển thị ứng dụng khi demo trên giả lập.

Ở phần này báo cáo cũng sẽ làm rõ quy trình xây dựng một ứng dụng Media trên Android Automotive OS, bao gồm cấu hình ứng dụng, khai báo các thành phần cần thiết trong hệ thống, xử lý luồng phát âm thanh và tích hợp với các cơ chế điều khiển Media của nền tảng.

III.3 Cấu trúc thư mục project

Project được tổ chức theo cấu trúc chuẩn của một ứng dụng Android. Các thư mục và file chính được đặt trong module **app**, trong đó phần quan trọng nhất nằm tại đường dẫn **app/src/main**. Cấu trúc tổng quan của project có thể mô tả như sau:

Cấu trúc thư mục hust_med_demo

```
hust_med_demo/
|-- .gradle/
|-- .idea/
|-- .kotlin/
|-- app/
|   |-- build/
|   '-- src/
|       |-- androidTest/
|       |-- test/
|       '-- main/
|           |-- AndroidManifest.xml
|           |-- java/kotlin/
|           |   '-- com/
|           |       '-- example/
|           |           '-- hust_med_demo/
|           |               |-- MyMediaService
|           |               '-- AutomotiveLibraryCallback
|           '-- res/
|               |-- drawable/
|               |-- layout/
|               |-- mipmap/
|               |-- values/
|               '-- xml/
|-- build/
|-- build.gradle
|-- settings.gradle
'-- gradle.properties
```

III.3.1 Thư mục app/src/main

Thư mục **app/src/main** là nơi chứa mã nguồn chính, tài nguyên giao diện và file cấu hình của ứng dụng. Đây là thành phần trung tâm trong quá trình phát triển ứng dụng Android, vì hầu hết các logic xử lý, giao diện và khai báo hệ thống đều được đặt trong thư mục này.

- **File `AndroidManifest.xml`** Đây là file cấu hình của ứng dụng Android, dùng để khai báo các thành phần như `activity`, `service`, quyền truy cập và các `metadata` cần thiết. Đối với project, file này có vai trò khai báo các thành phần phục vụ cho việc tích hợp ứng dụng media với môi trường Android Automotive OS.
- **Mã nguồn `Kotlin/Java`**: Đây là nơi chứa các file mã nguồn chính của ứng dụng, bao gồm các thành phần xử lý logic phát nhạc, kết nối với hệ thống media của Android Automotive OS và quản lý dữ liệu.
- **Thư mục `res`**: Thư mục này chứa các tài nguyên được sử dụng trong ứng dụng như hình ảnh, layout, biểu tượng, chuỗi văn bản, theme và các tài nguyên âm thanh nếu có.

III.3.2 File `AndroidManifest.xml`

File `AndroidManifest.xml` có vai trò khai báo các thông tin quan trọng của app với hệ điều hành Android, vì vậy đây là file cấu hình trung tâm.

Cụ thể, file `AndroidManifest.xml` trong project có thể đảm nhiệm các vai trò chính sau:

- Khai báo thông tin package của ứng dụng;
- Khai báo activity chính dùng để khởi chạy ứng dụng;
- Khai báo service phát nhạc, ví dụ như `MyMediaService`;
- Khai báo các quyền truy cập cần thiết cho ứng dụng;
- Khai báo metadata để hệ thống Android Automotive OS có thể nhận diện và tích hợp ứng dụng media.

File `AndroidManifest.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-feature
        android:name="android.hardware.type.automotive"
        android:required="true" />

    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission android:name="android.permission.
    FOREGROUND_SERVICE_MEDIA_PLAYBACK" />

    <application
        android:allowBackup="true"
```

```

        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Hust_Med_Demo">

        <service
            android:name=".MyMediaService"
            android:exported="true"
            android:foregroundServiceType="mediaPlayback">
            <intent-filter>
                <action android:name="androidx.media3.session.MediaLibraryService
" />
                <action android:name="android.media.browse.MediaBrowserService" /
>
            </intent-filter>
        </service>

        <meta-data
            android:name="com.google.android.gms.car.application"
            android:resource="@xml/automotive_app_desc" />

    </application>

</manifest>

```

1. Khai báo ứng dụng dành cho Android Automotive

File `AndroidManifest.xml`

```

<uses-feature
    android:name="android.hardware.type.automotive"
    android:required="true" />

```

- Đoạn này cho biết ứng dụng được thiết kế cho thiết bị có phần cứng dạng ô tô.
- `required = "true"` : ứng dụng chỉ chạy trên thiết bị Android Automotive. Nếu thiết bị không hỗ trợ automotive thì ứng dụng sẽ không được xem là tương thích.

2. Khai báo quyền chạy Foreground Service cho phát nhạc

File `AndroidManifest.xml`

```
<uses-permission
    android:name="android.permission.FOREGROUND_SERVICE" />

<uses-permission
    android:name="android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK" />
```

- Như đã nói phía trên, khi build Media-app ta cần sử dụng Foreground Service, vì vậy quyền đầu tiên khai báo: Cho phép ứng dụng sử dụng foreground service.
- Quyền thứ hai: Cho phép foreground service thuộc loại Media_Playback.

⇒ Đây là khai báo hợp lý vì ứng dụng media thường cần tiếp tục phát âm thanh ngay cả khi người dùng không mở trực tiếp giao diện ứng dụng.

3. Khai báo thông tin chung của ứng dụng

File `AndroidManifest.xml`

```
<application
    android:allowBackup="true"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.Hust_Med_Demo">
```

- `android:icon="@mipmap/ic_launcher"` : icon chính của ứng dụng.
- `android:label="@string/app_name"` : tên hiển thị của ứng dụng trên màn hình xe.
- `android:theme="@style/Theme.Hust_Med_Demo"` : xác định giao diện của ứng dụng.

4. Khai báo Media Service

File `AndroidManifest.xml`

```
<service
    android:name=".MyMediaService"
    android:exported="true"
```

```
android:foregroundServiceType="mediaPlayback">
```

Đây là phần quan trọng nhất trong File manifest.xml:

- `android:name=".MyMediaService"` : Khai báo service xử lý nhạc của ứng dụng. Service này có nhiệm vụ cung cấp nội dung để Android Automotive OS có thể duyệt danh sách bài hát, album, playlist hoặc điều khiển phát nhạc.
- `android:exported="true"` : Cho phép các thành phần bên ngoài ứng dụng, cụ thể là Android Automotive OS, có thể kết nối tới app.
- `android:foregroundServiceType="mediaPlayback"` : khai báo rằng foreground service này thuộc loại phát media. Điều này phù hợp với ứng dụng nghe nhạc hoặc phát âm thanh.

5. Khai báo intent-filter cho Media Browser Service

File `AndroidManifest.xml`

```
<intent-filter>
    <action android:name="androidx.media3.session.MediaLibraryService" />
    <action android:name="android.media.browse.MediaBrowserService" />
</intent-filter>
```

- Phần này khai báo các action mà service hỗ trợ:
 - Cho biết service sử dụng `MediaLibraryService` của thư viện `Media3` - hiện nay chủ yếu xây dựng trên thư viện này.
 - AAOS có thể phát hiện và kết nối tới media browser service của ứng dụng.

III.3.3 Các thư mục tài nguyên trong res

Thư mục `res` chứa các tài nguyên được sử dụng trong ứng dụng, bao gồm hình ảnh, biểu tượng, chuỗi văn bản, màu sắc, theme và các file cấu hình XML khác.

- `res/drawable`: chứa hình ảnh, icon hoặc các file drawable dùng trong giao diện. Vì vậy, ta sẽ đặt các ảnh minh họa cho bài hát vào đây.
- `res/mipmap`: chứa biểu tượng ứng dụng ở nhiều độ phân giải khác nhau.
- `res/values`: chứa các file cấu hình như `strings.xml`, `colors.xml`, `themes.xml`.

- **res/raw**: Folder này do ta tự thêm, dùng để chứa các file như mp3, mp4. Vì vậy, ta sẽ đặt các file bài hát vào đây.
- **res/xml**: có thể chứa các file cấu hình đặc thù cho ứng dụng media hoặc Android Automotive OS.

III.3.4 Các file mã nguồn trong main

1. File: `MyMediaService.kt`

Vai trò: Đây là thành phần dịch vụ nền `Background Service`, đóng vai trò là “trái tim” của ứng dụng Media. Nó kế thừa từ `MediaLibraryService` của thư viện `Media3` để quản lý lifecycle của trình phát nhạc và session.

File `MyMediaService.kt`

```
package com.example.hust_med_demo

import androidx.media3.exoplayer.ExoPlayer // Trình phát nhạc
import androidx.media3.session.MediaLibraryService
import androidx.media3.session.MediaSession // Bo kết nối với xe

class MyMediaService : MediaLibraryService() {

    private lateinit var player: ExoPlayer
    private lateinit var session: MediaLibrarySession

    override fun onCreate() {
        super.onCreate()
        player = ExoPlayer.Builder(this).build()
        session = MediaLibrarySession.Builder(
            this,
            player, // t player với y
            AutomotiveLibraryCallback()
        ).build()
    }

    override fun onGetSession(controllerInfo: MediaSession.ControllerInfo):
        MediaLibrarySession? {
        return session
    }
}
```

```

        override fun onDestroy() {
            session.release()
            player.release()
            super.onDestroy()
        }
    }
}

```

- **Các thành phần chính:**

- **ExoPlayer**: Trình phát đa phương tiện của Google được sử dụng để xử lý việc giải mã và phát các file âm thanh.
- **MediaLibrarySession**: Cung cấp giao diện để các ứng dụng bên ngoài như hệ thống Android Auto/Automotive có thể điều khiển Exoplayer như ta đã nói ở trên. Về cơ bản đây là **MediaSession**: nhận command, gửi playback-state cho **MediaController**

- **Chức năng chính:**

- **onCreate()**: Hàm này được override trong class này để khởi tạo **ExoPlayer** và thiết lập **MediaLibrarySession** cùng với **AutomotiveLibraryCallback**. Đây là nơi kết nối giữa **Exoplayer** và **MediaController**.
- **onGetSession()**: Hàm này cũng được override để cung cấp phiên làm việc hiện tại cho hệ thống khi có yêu cầu kết nối từ các Controller như màn hình điều khiển trên xe.
- **onDestroy()**: Đảm bảo giải phóng tài nguyên **player** và **session** khi dịch vụ bị dừng, tránh rò rỉ bộ nhớ.

2. File: **AutomotiveLibraryCallback.kt**

Vai trò: Đây là lớp xử lý logic điều hướng và cung cấp dữ liệu nhạc. Nó định nghĩa cấu trúc thư mục nhạc mà người dùng nhìn thấy trên màn hình ô tô và cách các file nhạc được tải lên. Code cụ thể đã

- **Các thành phần chính:**

- **Dữ liệu nhạc**: Quản lý danh sách các bài hát ví dụ: Kiếp Ve Sầu, Hoa Hồng, Con Mưa Tình Yêu... được lưu trữ trong thư mục **res/raw**.
- **Metadata**: Gắn kèm thông tin như tên bài hát, nghệ sĩ và ảnh bìa **artworkUri** từ thư mục **res/drawable**.

- **Chức năng quan trọng:**

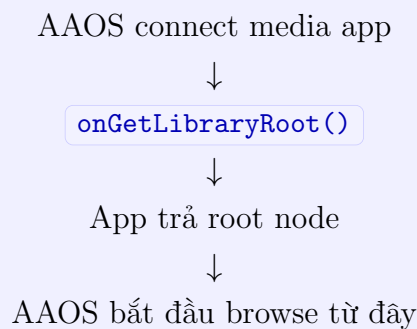
- `onGetLibraryRoot()` : Định nghĩa nút gốc của thư viện nhạc tên là “My Media”. Đây là điểm bắt đầu khi người dùng mở ứng dụng trên xe.

`onGetLibraryRoot`

```
override fun onGetLibraryRoot(
    session: MediaLibraryService.MediaLibrarySession,
    browser: MediaSession.ControllerInfo,
    params: MediaLibraryService.LibraryParams?
): ListenableFuture<LibraryResult<MediaItem>> {
    val root = MediaItem.Builder()
        .setMediaId("root")
        .setMediaMetadata(
            MediaMetadata.Builder()
                .setTitle("My Media")
                .setIsBrowsable(true) //day la folder/menu
                .setIsPlayable(false) //0 la leaf-node (song) ->
                không phát được
        )
        .build()
    return Futures.immediateFuture(LibraryResult.ofItem(root,
        params))
}
```

Workflow

★ Luồng hoạt động khi AAOS browse thư viện nhạc



- `onGetChildren()` : Sau khi có root, AAOS sẽ hỏi tiếp "Bên trong Root có gì?-> `onGetChildren()` được gọi. Hàm này xử lý phân cấp thư mục. Đây là hàm trả danh sách item nằm bên trong 1 folder

onGetLibraryRoot

```
override fun onGetChildren(  
    session: MediaLibraryService.MediaLibrarySession,  
    browser: MediaSession.ControllerInfo,  
    parentId: String,  
    page: Int,  
    pageSize: Int,  
    params: MediaLibraryService.LibraryParams?  
): ListenableFuture<LibraryResult<ImmutableList<MediaItem>>> {  
    val children = when (parentId) {  
        "root" -> listOf(  
            MediaItem.Builder()  
                .setMediaId("songs")  
                .setMediaMetadata(  
                    MediaMetadata.Builder()  
                        .setTitle("Songs")  
                        .setIsBrowsable(true)  
                        .setIsPlayable(false)  
                        .build()  
                ).build()  
        )  
        "songs" -> items  
        else -> emptyList()  
    }  
    return Futures.immediateFuture(LibraryResult.ofItemList(  
        ImmutableList.copyOf(children), params))  
    }
```

Workflow

★ Luồng hoạt động khi AAOS duyệt thư viện nhạc

AAOS connect media app



onGetLibraryRoot()



App trả folder gốc `root`



onGetChildren("root")



- `onAddMediaItems()` : Một bước quan trọng để giải mã các item. Vì trong AAOS, UI chỉ nhận **mediaId** và metadata.

Ta có:

Thành phần	Dùng để làm gì
mediaId	ID định danh một item nhạc
URI	đường dẫn thật đến file audio hoặc nguồn phát nhạc.

Bảng III.3: Phân biệt mediaId và URI trong MediaItem

Trong đó, AAOS UI chỉ nhận:

- * mediaId;
- * title;
- * artist;
- * artwork;
- * playable/browsable.

Workflow hiển thị bài hát



Ví dụ dễ hiểu

Giả sử có bài hát:

- * Tên bài: Nắng Ấm Xa Dàn
- * Ca sĩ: Sơn Tùng M-TP
- * File thật: `res/raw/nang_am_xa_dan.mp3`

Ta có thể lưu như sau:

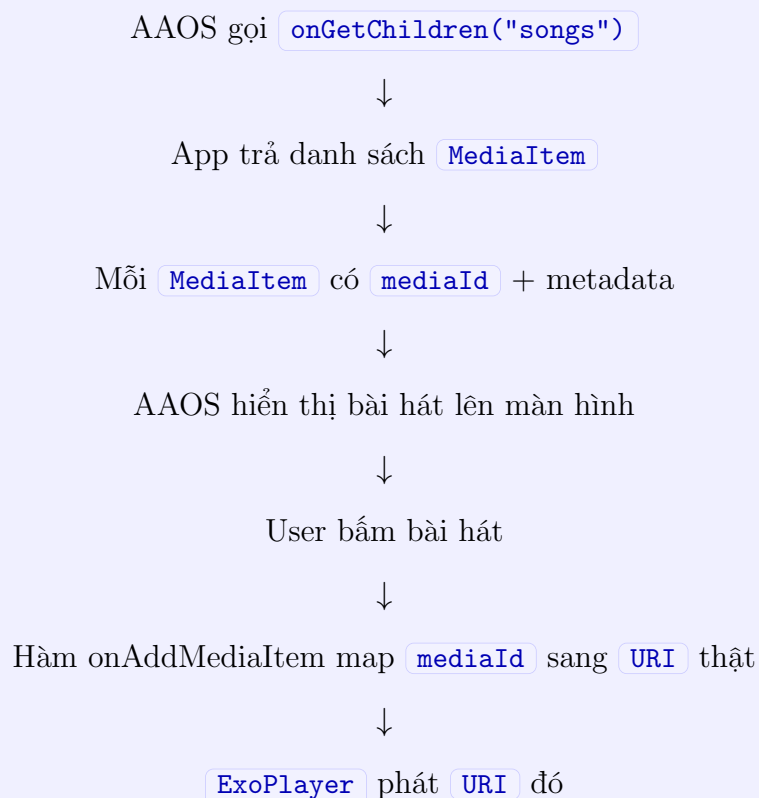
Ví dụ ánh xạ thông tin bài hát

```
mediaId = "song_nang_am_xa_dan"
uri = "android.resource://com.example.app/raw/nang_am_xa_dan"
title = "Nang Am Xa Dan"
artist = "Son Tung M-TP"
```

Nhận được MediaID và metadata trên sẽ hiển thị lên UI. Còn `ExoPlayer` mới là thành phần cần `Uri` để phát nhạc.

Workflow

★ Luồng phát nhạc từ danh sách bài hát



Code cụ thể Mapping

```
val resolved = mediaItems.map { item ->
    val uri = tracks[item.mediaId] //Map mediaID -> uri
```

Danh sách map

```
// Map mediaId -> URI file nhạc
private val tracks: Map<String, Uri> = mapOf(
    "track_1" to uri1,
    "track_2" to uri2,
    "track_3" to uri3,
    "track_4" to uri4,
    "track_5" to uri5
)

// Map mediaId -> URI ảnh bìa
private val artworks: Map<String, Uri> = mapOf(
    "track_1" to art1,
    "track_2" to art2,
    "track_3" to art3,
    "track_4" to art4,
    "track_5" to art5
)
```

với tracks là biến/property kiểu Map trong Kotlin

- `onGetItem()`: Trong AAOS, khi hệ thống đã biết mediaId của một bài hát và muốn lấy MediaItem của bài hát đó, nó sẽ gọi hàm này.

Thông tin	Ý nghĩa
<code>mediaId</code>	Mã định danh item
<code>title</code>	Tên bài hát / tên folder
<code>artist</code>	Nghệ sĩ
<code>albumTitle</code>	Tên album
<code>artworkUri</code>	Ảnh bìa
<code>isPlayable</code>	Có phát được không
<code>isBrowsable</code>	Có thể mở vào bên trong không

Bảng III.4: Một số thông tin thường có trong MediaItem

OnGetItem

```
override fun onGetItem(
    session: MediaLibraryService.MediaLibrarySession,
    browser: MediaSession.ControllerInfo,
    mediaId: String
): ListenableFuture<LibraryResult<MediaItem>> {
    val item = items.firstOrNull { it.mediaId == mediaId }
    return Futures.immediateFuture(
```

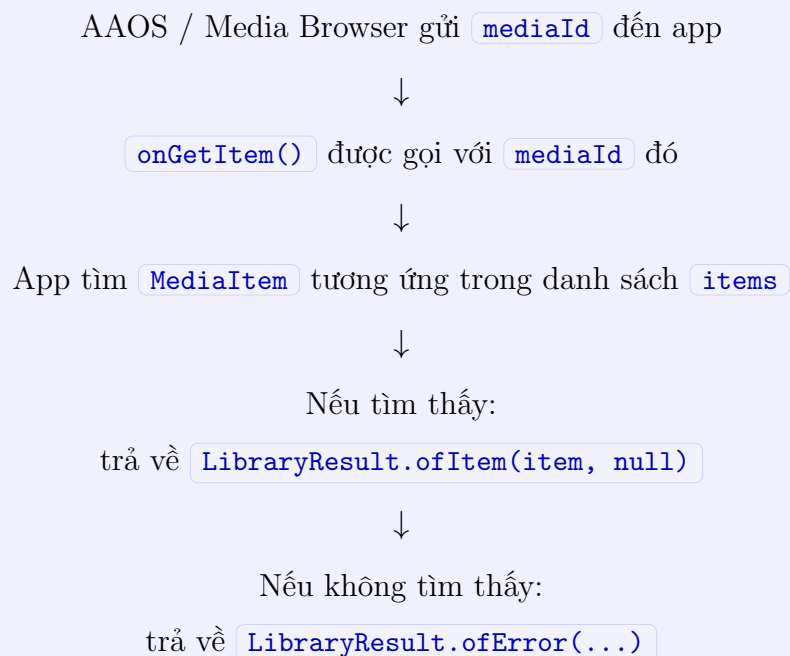
```

        if (item != null) {
            LibraryResult.ofItem(item, null)
        } else {
            LibraryResult.ofError(SessionError(SessionError.
ERROR_BAD_VALUE, "Unknown mediaId: $mediaId"))
        }
    )
}

```

Workflow

★ Luồng xử lý khi AAOS yêu cầu một MediaItem cụ thể



3. Cơ chế hoạt động phối hợp

1. Khi ứng dụng khởi chạy trên ô tô, `MyMediaService` được hệ thống gọi để thiết lập kết nối.
2. Hệ thống yêu cầu danh sách nhạc thông qua các hàm callback trong `AutomotiveLibraryCallback` để hiển thị lên màn hình UI.
3. Khi người dùng nhấn “Play” một bài hát, `AutomotiveLibraryCallback` sẽ cung cấp đường dẫn file URI cho `ExoPlayer` thông qua `onAddMediaItems()`.
4. `ExoPlayer` nằm trong `MyMediaService` sẽ tiến hành phát nhạc và phản hồi trạng thái thời gian, trạng thái play/pause ngược lại màn hình xe.

4. Demo

Sự kết hợp giữa `MyMediaService` và `AutomotiveLibraryCallback` giúp ứng dụng tuân thủ tiêu chuẩn của Android Automotive OS, cho phép người dùng điều khiển nhạc an toàn và thuận tiện thông qua giao diện chuẩn của ô tô mà không cần thao tác trực tiếp trên điện thoại.

Link demo project: <https://www.youtube.com/watch?v=KfEvP0I6PiE>

III.4 Các thành phần chính của ứng dụng

III.5 Luồng hoạt động của ứng dụng

III.6 Kết quả triển khai

III.7 Khó khăn và cách khắc phục

Chương IV. Ứng dụng thực tế và xu hướng phát triển

IV.1 Ứng dụng trong ngành công nghiệp ô tô

IV.2 Ưu điểm

IV.3 Hạn chế

IV.4 Xu hướng phát triển

Kết luận

Tài liệu tham khảo

- [1] Android Developers, *Android Automotive OS Documentation*. Available at: <https://source.android.com/docs/automotive>
- [2] Android Developers, *Android Auto Documentation*. Available at: <https://developer.android.com/training/cars>
- [3] Android Open Source Project, *Android Automotive Architecture*. Available at: https://source.android.com/docs/automotive/start/what_automotive